

Множества

В прошлых уроках мы изучили три типа коллекций в Python:

- списки – изменяемые коллекции элементов;
- строки – неизменяемые коллекции символов;

Следующий тип коллекций (наборов данных) – **множество**.

Множество – структура данных, организованная так же, как математические множества.

Важно знать:

- все элементы множества различны (уникальны), два элемента не могут иметь одинаковое значение;
- множества неупорядочены, то есть элементы не хранятся в каком-то определенном порядке;
- элементы множества должны относиться к неизменяемым типам данных;
- хранящиеся в множестве элементы могут иметь разные типы данных.

Структура данных (data structure) – программная единица, позволяющая **хранить и обрабатывать** множество однотипных и/или логически связанных данных.

Создание множества

Чтобы создать множество, нужно перечислить его элементы через запятую в фигурных скобках:

```
numbers = {2, 4, 6, 8, 10}
```

```
languages = {'Python', 'C#', 'C++', 'Java'}
```

Множество `numbers` состоит из 5 элементов, и каждый из них – **целое число**.

Множество `languages` состоит из 4 элементов, каждый из которых – **строка**.

Множества могут содержать значения **разных типов данных**:

```
info = {'Timur', 1992, 61.5}
```

Множество `info` содержит строковое значение, целое число и число с плавающей точкой.

Не создавайте переменные с именем `set`. Это очень плохая практика.

Пустое множество

Создать пустое множество можно с помощью встроенной функции, которая называется `set()`:

```
myset = set() # пустое множество
```

Обратите внимание — создать пустое множество с помощью пустых фигурных скобок нельзя:

```
myset = {} # создается словарь
```

С помощью пустых фигурных скобок создаются словари, так сложилось исторически, словари появились в Python раньше, чем множества.

Пустое множество создаётся исключительно через `set()`

Вывод множества

Для вывода всего множества можно использовать функцию `print()`:

```
numbers = {2, 4, 6, 8, 10}
languages = {'Python', 'C#', 'C++', 'Java'}
mammals = {'cat', 'dog', 'fox', 'elephant'}

print(numbers)
print(languages)
print(mammals)
```

Функция `print()` выводит на экран элементы множества, в фигурных скобках, разделенные запятыми:

```
{2, 4, 6, 8, 10}
{'C#', 'Python', 'Java', 'C++'}

{'dog', 'cat', 'fox', 'elephant'}
```

Обратите внимание: при выводе множества порядок элементов может отличаться от существовавшего при его создании, поскольку множества — неупорядоченные коллекции данных.

Встроенная функция set()

Встроенная функция `set()` помимо создания пустого множества может преобразовывать некоторые типы объектов в множества.

В функцию `set()` можно передать один аргумент. Передаваемый аргумент должен быть итерируемым объектом, таким как список, кортеж или строковое значение. Отдельные элементы объекта, передаваемого в качестве аргумента, становятся элементами множества:

```
myset1 = set(range(10))          # множество из элементов
# последовательности
myset2 = set([1, 2, 3, 4, 5])    # множество из элементов списка
myset3 = set('abcd')            # множество из элементов строки

myset4 = set((10, 20, 30, 40))  # множество из элементов кортежа
```

Пустое множество также можно создать передав функции `set()` в качестве аргумента пустой список, строку или кортеж:

```
emptyset1 = set([])             # пустое множество из пустого списка
emptyset2 = set('')             # пустое множество из пустой строки

emptyset3 = set(())             # пустое множество из пустого кортежа
```

Дубликаты при создании множеств

Множества не могут содержать повторяющиеся элементы. Если в функцию `set()` передать аргумент, содержащий повторяющиеся элементы, то в множестве появится только один из этих повторяющихся элементов.

Приведенный ниже код:

```
myset1 = {2, 2, 4, 6, 6}
myset2 = set([1, 2, 2, 3, 3])
myset3 = set('aaaaabbbbccccddd')

print(myset1)
print(myset2)

print(myset3)
```

выводит (порядок элементов может отличаться):

```
{2, 4, 6}  
{1, 2, 3}
```

```
{'b', 'c', 'd', 'a'}
```

Если требуется создать множество, в котором каждый элемент — строковое значение, содержащее более одного символа, то используем код:

```
myset = set(['aaa', 'bbbb', 'cc'])
```

```
print(myset)
```

Приведенный выше код выводит (порядок элементов может отличаться):

```
{'bbbb', 'aaa', 'cc'}
```

Если же создать множество следующим образом:

```
myset = set('aaa bbbb cc')
```

```
print(myset)
```

то мы получим (порядок элементов может отличаться):

```
{' ', 'c', 'a', 'b'}
```

Обратите внимание на наличие пробела в качестве элемента множества `myset`.

Примечания

Примечание 1. Элементы множества могут принадлежать любому **не изменяемому типу данных**, быть числами, строками, кортежами. Элементы изменяемых типов данных не могут входить в множества, в частности, нельзя сделать элементом множества список или другое множество. Требование неизменяемости элементов множества накладывается особенностями представления множеств в Python.

Приведенный ниже код:

```
myset1 = {1, 2, [5, 6], 7}    # множество не может содержать  
список
```

```
myset2 = {1, 2, {5, 6}, 7} # множество не может содержать
множество
```

приводит к ошибке:

```
TypeError: unhashable type: 'list'
```

```
TypeError: unhashable type: 'set'
```

Однако приведенный ниже код:

```
myset = {1, 2, (5, 6), 7} # множество может содержать кортеж
```

работает как полагается.

Основы работы с множествами

Работа с множествами очень сильно напоминает работу со списками, поскольку и множества, и списки содержат отдельные элементы, хотя элементы множества уникальны, а списки могут содержать повторяющиеся элементы. Многие из того, что мы делали со списками, доступно и при работе с множествами.

Функция len()

Длиной множества называется количество его элементов. Чтобы посчитать длину множества используют встроенную функцию `len()` (от слова length – длина).

Следующий программный код:

```
myset1 = {2, 2, 4, 6, 6}
myset2 = set([1, 2, 2, 3, 3, 4, 4, 5, 5])
myset3 = set('aaaaabbbbccccddd')

print(len(myset1))
print(len(myset2))

print(len(myset3))
```

выведет:

```
3
5
4
```

Оператор принадлежности in

Оператор `in` позволяет проверить, содержит ли множество некоторый элемент.

Рассмотрим следующий код:

```
numbers = {2, 4, 6, 8, 10}

if 2 in numbers:
    print('Множество numbers содержит число 2')
else:
    print('Множество numbers не содержит число 2')
```

Такой код проверяет, содержит ли множество `numbers` число `2` и выводит соответствующий текст:

```
Множество numbers содержит число 2
```

Мы можем использовать оператор `in` вместе с логическим оператором `not`. Например

```
numbers = {2, 4, 6, 8, 10}

if 0 not in numbers:
    print('Множество numbers не содержит нулей')
```

Оператор принадлежности `in` работает **очень быстро** на множествах, намного быстрее чем на списках. Поэтому если требуется часто осуществлять поиск в коллекции уникальных данных, то множество – подходящий выбор.

Встроенные функции sum(), min(), max()

Встроенная функция `sum()` принимает в качестве аргумента множество чисел и вычисляет сумму его элементов.

Следующий программный код:

```
numbers = {2, 2, 4, 6, 6}

print('Сумма всех элементов множества =', sum(numbers))
```

ВЫВОДИТ:

```
Сумма всех элементов множества = 12
```

Встроенные функции `min()` и `max()` принимают в качестве аргумента множество и находят минимальный и максимальный элементы соответственно.

Следующий программный код:

```
numbers = {2, 2, 4, 6, 6}
print('Минимальный элемент =', min(numbers))

print('Максимальный элемент =', max(numbers))
```

ВЫВОДИТ:

```
Минимальный элемент = 2
```

```
Максимальный элемент = 6
```

Примечания

Примечание 1. Индексация и срезы **недоступны** для множеств.

Примечание 2. Операция конкатенации `+` и умножения на число `*` **недоступны** для множеств.

Перебор элементов множества

Перебор элементов множества осуществляется точно так же, как и перебор элементов списка, то есть с помощью цикла `for`.

Для вывода элементов множества **каждого на отдельной строке** можно использовать следующий код:

```
numbers = {0, 1, 1, 2, 3, 3, 3, 5, 6, 7, 7}

for num in numbers:

    print(num)
```

Такой код выведет (порядок элементов может отличаться):

```
0
1
2
3
5
6
```

7

Мы также можем использовать операцию **распаковки множества**.

Приведенный ниже код:

```
numbers = {0, 1, 1, 2, 3, 3, 3, 5, 6, 7, 7}
```

```
print(*numbers, sep='\n')
```

выводит (порядок элементов может отличаться):

0

1

2

3

5

6

7

Не стоит забывать, что множества – неупорядоченные коллекции, поэтому полагаться на порядок вывода элементов не стоит. Если нужно гарантировать порядок вывода элементов (по возрастанию / убыванию), то необходимо воспользоваться встроенной функцией `sorted()`.

Приведенный ниже код:

```
numbers = {0, 1, 1, 2, 3, 3, 3, 5, 6, 7, 7}
```

```
sorted_numbers = sorted(numbers)
```

```
print(*sorted_numbers, sep='\n')
```

будет **гарантированно** выводить элементы множества в порядке возрастания.

Обратите внимание на то, что функция `sorted()` возвращает отсортированный список, а не множество. Не путайте встроенную функцию `sorted()` и списочный метод `sort()`. Множества **не содержат** метода `sort()`.

Сравнение множеств

Множества можно сравнивать между собой. Равные множества имеют одинаковую длину и содержат равные элементы. Для сравнения множеств используются операторы `==` и `!=`.

Приведенный ниже код:

```
myset1 = {1, 2, 3, 3, 3, 3}
myset2 = {2, 1, 3}
myset3 = {1, 2, 3, 4}

print(myset1 == myset2)
print(myset1 == myset3)

print(myset1 != myset3)
```

ВЫВОДИТ:

```
True
False

True
```

Примечания

Примечание 1. Встроенная функция `sorted()` имеет опциональный параметр `reverse`. Если установить этот параметр в значение `True`, произойдет сортировка по убыванию.

Приведенный ниже код:

```
numbers = {0, 1, 1, 2, 3, 3, 3, 5, 6, 7, 7}

sortnumbers = sorted(numbers, reverse=True)

print(*sortnumbers, sep='\n')
```

гарантированно выводит:

```
7
6
5
3
2
1
```

Примечание 2. Код для работы с множествами нужно писать так, чтобы результат его выполнения не зависел от расположения элементов и был одинаковым при любом порядке обхода, последовательного обращения ко всем элементам.